

Lab0:GitLab 实验报告

问题一：认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。（15 分）

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

有多人协同开发的经历。我们在 GitHub 上建立了共享仓库，成员接受邀请后加入该仓库，再上传或下载项目文件。然而，我们之前对 GitHub 的使用较简单，主要是在网页上直接上传或下载文件，没有系统使用本地 Git 的暂存、提交、分支和合并功能。

2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

“暂存”是指使用 `git add` 命令，把指定文件放入暂存区。“提交”是指使用 `git commit` 命令，将暂存区中的内容保存到 Git 仓库。

设立两个步骤使开发者得以筛选与检查即将提交的内容。此外，开发者可以从很多内容中挑选相关部分，组成一次目标明确的提交，使提交历史更清晰，便于之后查看、回退和与他人协作。

3. `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

`git branch` 只显示本地分支，而 `git branch -a` 同时显示本地分支和远程跟踪分支。其中，远程跟踪分支是本地仓库对远程分支状态的记录，不一定与远程仓库的最新状态完全一致。通常需要先执行 `git fetch` 更新远程仓库的信息，再使用 `git branch -a` 查看。

问题二：使用我们的模板仓库建立个人仓库，完成 `main.c` 文件中的 TODO 部分并进行一次 `commit`。（50 分）

我通过课程提供的模板仓库创建了个人仓库，随后使用 SSH 将个人仓库克隆到了本地。

```
f10xz@XINZI-T14P:~/ics-labs$ git clone git@github.com:muyuhub0512/lab0-gitlab.git
Cloning into 'lab0-gitlab'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 0), reused 7 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.
f10xz@XINZI-T14P:~/ics-labs$ cd lab0-gitlab
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ ls
git status
Makefile  README.md  main.c
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

然后，在 VS Code 中打开仓库并修改 main.c，完成其中的 TODO，将程序的输出内容修改为 “Hello, GitLab!”。

```
C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, GitLab!\n");
7  }
8
```

程序成功编译并输出。

```
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ make
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ ./main
Hello, GitLab!
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ make clean
rm -f main.o main
```

随后，提交修改：

```
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git add main.c
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git commit -m "feat: complete main.c TODO"
[main 4159243] feat: complete main.c TODO
1 file changed, 2 insertions(+), 2 deletions(-)
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 22 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 303 bytes | 303.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:muyuhub0512/lab0-gitlab.git
a72e828..4159243 main -> main
branch 'main' set up to track 'origin/main'.
f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$
```

问题三：在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。（15 分）

1. Commit Message 规范

Commit Message 是每次执行 git commit 时填写的提交说明，用于解释这次提交的目的。规范、清晰的提交信息能够形成更容易阅读与查找的项目历史。

Commit Message 由 Header、Body 和 Footer 三部分组成，其中 Header 必须填写。基本提交格式如下：

```
<type>(<scope>): <subject>
<body>
<footer>
```

Header 中:

①type 表示提交类型, 例如 feat 表示增加功能, fix 表示修复错误, docs 表示修改文档, style 表示不影响程序运行的格式调整, refactor 表示代码重构, test 表示增加或修改测试, chore 表示构建过程或辅助工具的修改。

②scope 表示本次修改影响的范围, 可以省略。

③subject 是对提交目的的简短说明。

Body 用于详细说明修改的原因和修改前后的差别。

footer 主要用于说明不兼容的改动或关联、关闭某个 Issue。

如果所有提交都遵守统一格式, 还可以使用工具自动检查提交信息, 并根据提交记录生成 Change Log。

2. Git Flow 分支控制

Git Flow 是一种对 Git 分支进行分类和管理的工作流程, 它为不同分支规定了不同的用途。

Git Flow 中主要包括以下分支:

①master 或 main 是长期存在的主分支, 用于保存已经发布或可以正式发布的稳定代码。

②develop 是长期存在的开发分支, 用于汇总即将进入下一版本的开发成果。

③feature 是功能分支, 通常从 develop 创建。开发者在该分支中完成某项功能, 完成后再合并回 develop。

④release 是发布分支, 在准备发布新版本时从 develop 创建, 主要用于测试、调整版本号和修复发布前发现的问题。准备完成后再合并到主分支和开发分支。

⑤bugfix 是缺陷修复分支, 用于修复测试阶段发现的问题, 完成后合并回相应的发布分支。

⑥hotfix 是紧急修复分支。当已经发布的版本出现严重问题时, 从主分支或对应的版本标签创建, 修复后再合并回主分支和开发分支。

这一控制的核心是让更多类型的工作在各自的分支中完成, 并遵循“从相应分支创建, 完成后再合并回相应分支”的原则。这样可以保持主分支稳定, 也能让多个成员并行开发不同功能。Git Flow 适合成员较多、具有明确测试与发布阶段的项目, 对于非常简单的小型项目, 可能不需要使用完整的 Git Flow。这些操作既可以通过普通

Git 命令完成，也可以使用 Git Flow 扩展库简化。

3. 语义化版本

语义化版本是一套使用版本号表达软件变化的规则。标准版本号采用主版本号. 次版本号. 修订号的形式：

①主版本号：当软件的公共 API 出现不向下兼容的修改时递增，同时将次版本号和修订号归零。

②次版本号：当软件增加向下兼容的新功能，或者将某项功能标记为弃用时递增，同时将修订号归零。

③修订号：当软件只进行了向下兼容的问题修复时递增。

在正式形成稳定的公共 API 之前，可以使用 0.y.z 表示软件仍处于初始开发阶段；1.0.0 通常表示软件已经形成稳定的公共 API。版本号后还可以添加先行版本标识和构建信息，例如通过连接号表示测试版本，通过加号添加构建信息。

语义化版本使版本号在表示发布顺序的同时，说明新版本是否增加功能、修复问题或包含不兼容修改。

4. 为什么学习 Git

Git 提供了一套管理项目变化和多人协作的方法。

Commit Message 规范使每次修改的目的更加清楚；Git Flow 通过划分不同用途的分支，使开发、测试、发布和修复过程相互隔离；语义化版本则通过统一的版本号规则说明软件发生了怎样的变化。

在简单项目中，直接使用 GitHub 仓库可能已经能够满足基本需求。但当项目规模扩大或者多人同时开发时，就需要更加规范的提交记录、分支管理和版本发布方法。学习 Git 可以使项目历史更加清晰，让不同成员并行完成任务，并在出现问题时追踪修改来源或恢复之前的版本，因此对复杂的开发和团队协作具有重要意义。

问题四：学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交（10 分）。随后将 feature 分支 merge 到 main 分支（即切换回 main 分支执行 git merge feature），并处理发生的合并冲突（10 分）。

创建并切换到 feature 分支，在 main 和 feature 分支中修改 main.c 的同一条输出语句，并在两个分支上分别完成提交和推送。

```

● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git add main.c
git commit -m "feat: change message on feature"
git push origin feature
[feature 7a56849] feat: change message on feature
 1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 22 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 584 bytes | 584.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 2 local objects.
To github.com:muyuhub0512/lab0-gitlab.git
 4159243..7a56849 feature -> feature
● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
○ f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$

```

接下来，我将 feature 分支合并到 main 分支。由于两个分支都修改了 main.c 的同一位置，Git 无法自动判断应该采用哪一项内容，因此提示 main.c 出现合并冲突。

The image shows a code editor with a merge conflict in `main.c`. The conflict is between the HEAD (Current Change) and the feature branch (Incoming Change). The HEAD version has a comment `// @TODO: print a sentence you want.` and the feature version has `printf("Github!\n");`. The editor highlights the conflicting lines with green and blue backgrounds.

Below the code editor is a terminal window showing the following commands and output:

```

● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git switch feature
Already on 'feature'
Your branch is ahead of 'origin/feature' by 1 commit.
(use "git push" to publish your local commits)
● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git add main.c
git commit -m "feat: change message on feature"
git push origin feature
[feature 7a56849] feat: change message on feature
 1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 22 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 584 bytes | 584.00 KiB/s, done.
Total 6 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 2 local objects.
To github.com:muyuhub0512/lab0-gitlab.git
 4159243..7a56849 feature -> feature
● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
○ f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$

```

选择保留双方的输出语句并保存文件后，将解决结果提交并推送到远程仓库。随后重新编译和运行程序，程序能够正常输出两条信息，说明冲突已经解决。

```
● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ make
./main
make clean
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
Welcome to Github!
Github!
rm -f main.o main
○ f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$
```

最后，查看仓库的分支与提交历史。结果显示 main 和 feature 分支分别包含自己的修改记录，并最终汇合到一次合并提交中，说明分支合并已经完成。

```
● f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$ git log --graph --oneline --all --decorate
* 40193b2 (HEAD -> main, origin/main, origin/HEAD) merge: resolve main.c conflict
| \
| * 7a56849 (origin/feature, feature) feat: change message on feature
| * 3ada521 feat: update message on main
| * | 664e27f feat: update message on main
| /
* 4159243 feat: complete main.c TODO
* a72e828 Initial commit
○ f10xz@XINZI-T14P:~/ics-labs/lab0-gitlab$
```